

```
!pip install -q transformers datasets peft accelerate

!pip uninstall -y torchao peft transformers accelerate datasets
!pip install -q peft==0.13.2 transformers==4.45.2 accelerate==0.34.2 datasets==3.0.1

Found existing installation: torchao 0.10.0
Uninstalling torchao-0.10.0:
  Successfully uninstalled torchao-0.10.0
Found existing installation: peft 0.19.1
Uninstalling peft-0.19.1:
  Successfully uninstalled peft-0.19.1
Found existing installation: transformers 5.0.0
Uninstalling transformers-5.0.0:
  Successfully uninstalled transformers-5.0.0
Found existing installation: accelerate 1.13.0
Uninstalling accelerate-1.13.0:
  Successfully uninstalled accelerate-1.13.0
Found existing installation: datasets 4.0.0
Uninstalling datasets-4.0.0:
  Successfully uninstalled datasets-4.0.0
----- 44.4/44.4 kB 1.9 MB/s eta 0:00
----- 320.7/320.7 kB 10.1 MB/s eta 0:00
----- 9.9/9.9 MB 77.2 MB/s eta 0:00
----- 324.4/324.4 kB 19.3 MB/s eta 0:00
----- 471.6/471.6 kB 37.7 MB/s eta 0:00
----- 177.6/177.6 kB 18.3 MB/s eta 0:00
----- 566.4/566.4 kB 41.1 MB/s eta 0:00
----- 3.0/3.0 MB 76.3 MB/s eta 0:00
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This
causes pip to recommend installing packages that are already installed, resulting in duplicate packages.
ERROR: 2025.3.0 requires fsspec==2025.3.0, but you have fsspec 2024.6.1 which is incompatible.
```

```
import os
import json
import torch
from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForCausalLM,
    TrainingArguments,
    Trainer,
    DataCollatorForLanguageModeling,
)
from peft import LoraConfig, get_peft_model

print("PyTorch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())

PyTorch version: 2.10.0+cu128
CUDA available: True
```

```
train_examples = [
    {
        "instruction": "What is LoRA?",
        "response": "LoRA is a parameter-efficient fine-tuning method that freezes the original model weights"
    },
    {
        "instruction": "Why is LoRA useful for LLM fine-tuning?",
        "response": "LoRA is useful because it reduces the number of trainable parameters, lowers memory usage"
    },
    {
        "instruction": "Explain parameter-efficient fine-tuning.",
        "response": "Parameter-efficient fine-tuning adapts a pretrained model to a new task by updating only"
    },
    {
        "instruction": "What is the advantage of freezing the base model?",
        "response": "Freezing the base model saves memory and computation because the original pretrained para"
    },
    {
        "instruction": "Explain LoRA in simple Korean.",
        "response": "LoRA는 거대한 언어 모델 전체를 다시 학습하지 않고, 작은 추가 행렬만 학습해서 모델을 효율?"
    }
]

data_path = "train.jsonl"

with open(data_path, "w", encoding="utf-8") as f:
    for example in train_examples:
        f.write(json.dumps(example, ensure_ascii=False) + "\n")

print("train.jsonl exists:", os.path.exists(data_path))
print("Number of examples:", len(train_examples))

train.jsonl exists: True
Number of examples: 5
```

```

model_name = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
output_dir = "./tinyllama_lora_cpu"

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

if device.type == "cpu":
    torch.set_num_threads(4)

print("Device:", device)

tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float32 if device.type == "cpu" else torch.float16,
)

model.to(device)
print("Model loaded.")

```

```

Device: cuda
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings)
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
config.json: 100%  608/608 [00:00<00:00, 9.91kB/s]
tokenizer_config.json:  1.29k/? [00:00<00:00, 29.6kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate !
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please se
tokenizer.json:  1.84M/? [00:00<00:00, 15.8MB/s]
tokenizer.model: 100%  500k/500k [00:01<00:00, 2.52MB/s]
special_tokens_map.json: 100%  551/551 [00:00<00:00, 14.1kB/s]
`torch_dtype` is deprecated! Use `dtype` instead!
model.safetensors: 100%  2.20G/2.20G [00:15<00:00, 164MB/s]
Loading weights: 100%  201/201 [00:05<00:00, 18.18it/s, Materializing param=model.norm.weight

```

```

def generate_response(model, tokenizer, instruction, max_new_tokens=80):
    model.eval()

    prompt = (
        f"<|system|>\nYou are a helpful assistant.\n"
        f"<|user|>\n{instruction}\n"
        f"<|assistant|>\n"
    )

    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)

    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False,
            pad_token_id=tokenizer.eos_token_id,
        )

    return tokenizer.decode(outputs[0], skip_special_tokens=True)

test_instruction = "What is LoRA?"
print(generate_response(model, tokenizer, test_instruction))

```

```

<|system|>
You are a helpful assistant.
<|user|>
What is LoRA?
<|assistant|>
LoRA (Low Power Wide Area Network) is a low-power, long-range wireless communication technology that uses radio

```

```

lora_config = LoraConfig(
    r=4,
    lora_alpha=8,
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM",
    target_modules=["q_proj", "v_proj"],
)

model = get_peft_model(model, lora_config)
model.print_trainable_parameters()

```

```

*** trainable params: 563,200 || all params: 1,100,611,584 || trainable%: 0.0512

```

```

dataset = load_dataset("json", data_files={"train": data_path})["train"]

def format_example(example):
    text = (
        f"<|system|>\nYou are a helpful assistant.\n"
        f"<|user|>\n{example['instruction']}\n"
        f"<|assistant|>\n{example['response']}"
    )
    return {"text": text}

dataset = dataset.map(format_example)

print(dataset[0]["text"])

```

```

*** Generating train split: 5/0 [00:00<00:00, 132.61 examples/s]

Map: 100% 5/5 [00:00<00:00, 176.63 examples/s]
<|system|>
You are a helpful assistant.
<|user|>
What is LoRA?
<|assistant|>
LoRA is a parameter-efficient fine-tuning method that freezes the original model weights and trains small low-r

```

```

MAX_LEN = 128

def tokenize(example):
    out = tokenizer(
        example["text"],
        truncation=True,
        padding="max_length",
        max_length=MAX_LEN,
    )
    out["labels"] = out["input_ids"].copy()
    return out

tokenized_dataset = dataset.map(tokenize, remove_columns=dataset.column_names)

print(tokenized_dataset[0].keys())

```

```

*** Map: 100% 5/5 [00:00<00:00, 163.63 examples/s]
dict_keys(['input_ids', 'attention_mask', 'labels'])

```

```

training_args = TrainingArguments(
    output_dir=output_dir,
    per_device_train_batch_size=1,
    gradient_accumulation_steps=1,
    num_train_epochs=1,
    learning_rate=5e-4,
    logging_steps=1,
    save_steps=20,
    save_total_limit=1,
    fp16=False,
    bf16=False,
    report_to="none",
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset,
    data_collator=DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False),
)

```

```

train_result = trainer.train()

model.save_pretrained(output_dir)
tokenizer.save_pretrained(output_dir)

print("Done. Saved to:", output_dir)
print(train_result)

```

 [5/5 00:02, Epoch 1/1]

Step	Training Loss
1	2.139374
2	1.556549
3	2.233129
4	2.725815
5	2.284409

Done. Saved to: ./tinylama_lora_cpu

TrainOutput(global_step=5, training_loss=2.187855052947998, metrics={'train_runtime': 3.1458, 'train_samples_per_gpu': 5, 'train_steps_per_epoch': 5})

```

def questions = [
    "What is LoRA?",
    "Why is LoRA useful for LLM fine-tuning?",
    "Explain parameter-efficient fine-tuning.",
    "Explain LoRA in simple Korean."
]

for question in questions:
    print(f"> {question}")
    print(Generate.from_model(model, tokenizer, question))

```

Question: What is LoRA?

<Response>

You are a helpful assistant.

<User>

What is LoRA?

<Response>

LoRA is a shortened form of Low-Rank Adaptation Network (LoRA). It is a wireless communication technology that uses low power and short range to provide reliable and secure connectivity for IoT devices. It is used for short-range communication between devices, such as sensors, smart meters, and smart home devices. LoRA is a part of the LoRaWAN.

Question: Why is LoRA useful for LLM fine-tuning?

<Response>

You are a helpful assistant.

<User>

Why is LoRA useful for LLM fine-tuning?

<Response>

LoRA (Low-Rank Adaptation) is a technique used for fine-tuning a language model (LLM) for specific tasks. In LLM fine-tuning, the LLM is trained on a large corpus of data, and the task is to improve the LLM's performance on a specific task. LoRA is particularly useful for LLM fine-tuning.

Question: Explain parameter-efficient fine-tuning.

<Response>

You are a helpful assistant.

<User>

Explain parameter-efficient fine-tuning.

<Response>

Parameter-efficient fine-tuning is a technique that reduces the computational complexity of neural network models by optimizing the weights and biases of the model during training. This technique is particularly useful when training deep neural networks, as the number of parameters in the model can become very large.

In parameter-efficient fine-tuning, the model is trained on a smaller dataset, called the source

Question: Explain LoRA in simple Korean.

<Response>

You are a helpful assistant.

<User>

Explain LoRA in simple Korean.

<Response>

LoRA (Low-Rank Adaptation Network) is a wireless communication technology that uses low-power, long-range radio waves to connect devices. It is designed to provide low latency, high reliability, and low power consumption, making it ideal for IoT (Internet of Things) applications. In simple Korean, LoRA is called "LoRa".